

CSC 212: Data Structures and Abstractions

04: Asymptotic Notation (O, Ω, Θ)

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



Where we are?

- Last lecture we learned to count

- ✓ today we learn what to throw away

- By counting, we can produce exact costs:

$$5n + 5 \quad \frac{n(n-1)}{2} \quad \frac{n+1}{2} \quad 3n^2 + 100n + 500$$

- Every constant there is an artifact of our bookkeeping

- ✓ none survives a change of compiler, machine, or counting convention

- **Today:** a notation that discards the constant factors and keeps the essential

2

Motivation

2-sum

- Problem

- ✓ given an array A of n integers and a target tgt , decide whether two elements of A sum to tgt

0	1	2	3	4	5	6	7
4	3	-5	0	9	-2	7	1

- Three solutions, different costs

- ✓ brute force

- ✓ sort, then two pointers

- ✓ use hash tables

4

2-sum (brute force)

```
bool two_sum_bf(const std::vector<int>& A, int tgt) {
    for (size_t i = 0; i < A.size(); ++i) {
        for (size_t j = i + 1; j < A.size(); ++j) {
            if (A[i] + A[j] == tgt) {
                return true;
            }
        }
    }
    return false;
}
```

- Worst case

- ✓ no pair sums to tgt , every pair is examined once

$$T(n) = (n-1) + (n-2) + \dots + 1 + 0 = \sum_{i=0}^{n-1} i = \frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

- Best case

- ✓ one comparison

- Auxiliary space

- ✓ $\approx c$, constant

5

2-sum (sort, then two pointers)

```
bool two_sum_tp(const std::vector<int>& A, int tgt) {
    const size_t n = A.size();
    if (n < 2) return false;
    std::vector<int> B = A;
    std::sort(B.begin(), B.end());
    size_t left = 0, right = n - 1;
    while (left < right) {
        int current_sum = B[left] + B[right];
        if (current_sum == tgt) return true;
        else if (current_sum < tgt) left++;
        else right--;
    }
    return false;
}
```

- Worst case

- ✓ each step eliminates one element, the loop runs at most $n - 1$ times

$$T(n) = \underbrace{n \log n}_{\text{sort}} + \underbrace{n - 1}_{\text{scan}}$$

- Auxiliary space

- ✓ $\approx n$, a copy is needed

6

2-sum (using a hash table)

```
bool two_sum_hash(const std::vector<int>& A, int tgt) {
    std::unordered_set<int> seen;
    for (int x : A) {
        if (seen.count(tgt - x)) return true; // constant
        seen.insert(x); // constant
    }
    return false;
}
```

- One pass

- ✓ for each x , ask whether its complement $tgt - x$ has already been seen

$$T(n) = n \cdot \Theta(1) = \Theta(n)$$

hash tables provide expected $\Theta(1)$ operations

- Auxiliary space

- ✓ $\approx n$, the hash table

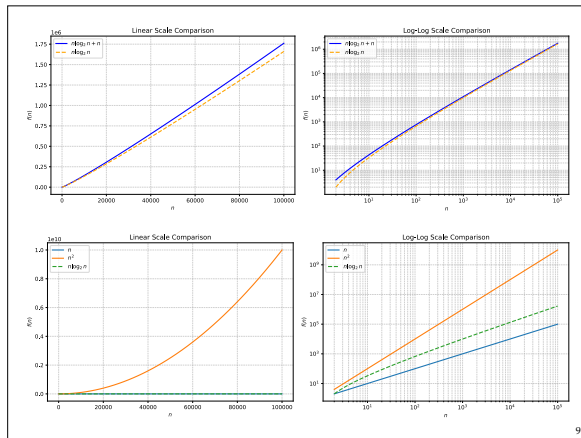
7

Comparing

Solution	Worst-case cost	Aux. space	Requires
Brute force	quadratic	constant	nothing
Sort and two pointers	linearithmic	linear	comparable keys
Hash table	linear	linear	hashable keys

Which is better?

8



9

Order of growth for different input sizes

Size	$T(n) = \log n$	$T(n) = n$	$T(n) = n \log n$	$T(n) = n^2$	$T(n) = n^3$
1	0	1	0	1	1
10	3	10	33	100	1,000
100	7	100	664	10,000	1,000,000
1,000	10	1,000	9,966	1,000,000	1,000,000,000
10,000	13	10,000	132,877	100,000,000	1,000,000,000,000 4 mins
100,000	17	100,000	1,660,964	10,000,000,000	1,000,000,000,000,000 3 days
1,000,000	20	1,000,000	19,931,569	1,000,000,000,000	1,000,000,000,000,000,000 8 years
10,000,000	23	10,000,000	232,534,967	100,000,000,000,000	1,000,000,000,000,000,000,000 7900 years

rounded

rounded

assume a basic 4Ghz processor

10

Q4-1

You must solve 2-sum on $n = 1,000,000$ integers. Your machine performs about 10^9 simple operations per second. Roughly how long does the brute-force solution take in the worst case?

- A. About 0.5 milliseconds
- B. About 0.5 seconds
- C. About 8 minutes
- D. About 12 days
- E. None of the above

11

The seven functions

The seven functions

Function	Name	Example algorithm(s)
1	Constant	array element access, push/pop on stack
$\log n$	Logarithmic	binary search, height of a balanced binary tree
n	Linear	linear search, traversing a list
$n \log n$	Linearithmic	merge sort, heap sort
n^2	Quadratic	bubble/insertion/selection sort, all-pairs loops
n^3	Cubic	naive matrix multiplication
$a^n, a > 1$	Exponential	subset enumeration, naive recursive Fibonacci

2^n with $n = 100$, at 1 nanosecond per operation: $2^{100} = 1.27 \times 10^{30} \text{ ops} \Rightarrow 1.27 \times 10^{21} \text{ s} \approx 4 \times 10^{13} \text{ years}$
that is, **40 trillion years** $\approx 2,900 \times$ the age of the universe

No hardware improvement rescues an exponential algorithm.
No amount of GPU scaling changes an asymptotic class.

Almost every algorithm you meet falls into one of seven growth classes. If you can place an algorithm in this table, you can usually predict its behavior before writing a line of code.

13

Q4-2

Order these from slowest-growing to fastest-growing:

$n \log n$ 2^n n^2 $\log n$ n $n!$ 1

- A. 1, $\log n$, n , $n \log n$, n^2 , 2^n , $n!$
B. 1, $\log n$, n , n^2 , $n \log n$, 2^n , $n!$
C. 1, $\log n$, n , $n \log n$, n^2 , $n!$, 2^n
D. $\log n$, 1, n , $n \log n$, n^2 , 2^n , $n!$
E. None of the above

14

Asymptotic notation

Intuition

For large n , the highest-order term dominates everything else

consider $T(n) = 3n^2 + 100n + 500$

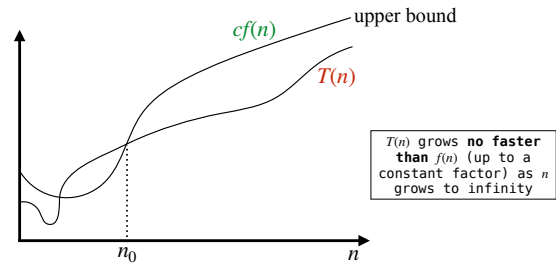
n	$3n^2$	$100n$	$T(n)$	$3n^2 / T(n)$
10	300	10^3	1,800	0.16
100	3×10^4	10^4	40,500	0.74
1,000	3×10^6	10^5	3.1×10^6	0.97
10,000	3×10^8	10^6	3.01×10^8	0.997

At $n = 10$ the linear term is bigger than the quadratic one. By $n = 1000$ the quadratic term is 97% of the total and the constant has vanished

In practice: drop constant factors, drop lower-order terms
 $3n^2 + 100n + 500 \rightarrow n^2$

16

Big O



$T(n)$ is $O(f(n)) \iff \exists c > 0, n_0 \geq 1$ such that $T(n) \leq cf(n) \quad \forall n \geq n_0$

17

Worked proof

• Claim: $T(n) = 3n^2 + 5n$ is $O(n^2)$

• Goal:

• exhibit $c > 0$ and $n_0 \geq 1$ with $3n^2 + 5n \leq cn^2$ for all $n \geq n_0$

• Proof:

For $n \geq 1$: $n \leq n^2 \Rightarrow 5n \leq 5n^2$

Therefore $3n^2 + 5n \leq 3n^2 + 5n^2 = 8n^2 \quad \forall n \geq 1$

Take $c = 8, n_0 = 1$

The pairs are not unique. $c = 9, n_0 = 1$ works. So does $c = 4, n_0 = 5$. You need only **one** pair

18

Q4-3

• A student claims $T(n) = 2n^2 + 3n$ is $O(n^2)$, and proves it with $c = 2, n_0 = 1$. Is the proof valid?

- A. Yes, 2 is the coefficient of the n^2 term, so $c = 2$ is correct
- B. No, you must always use $c = 1$
- C. No, $2n^2 + 3n \leq 2n^2$ fails for every $n \geq 1$, c must exceed 2
- D. No, the claim is false, $2n^2 + 3n$ is not $O(n^2)$
- E. None of the above

19

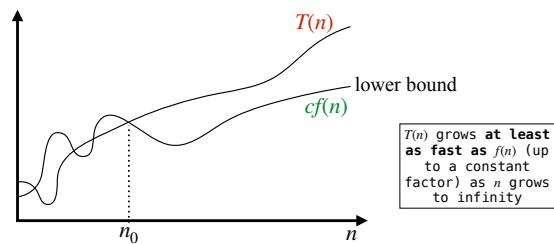
Q4-4

• Is $T(n) = 10^{21} + 3000n$ in $O(n^2)$?

- A. No, the constant 10^{21} is far too large
- B. No, $T(n)$ is linear, and linear is not $O(n^2)$
- C. Yes, and it is also $O(n)$, $O(n \log n)$, and $O(2^n)$
- D. Yes, but only for $n > 10^{21}$
- E. None of the above

20

Big Omega



$T(n)$ is $\Omega(f(n)) \iff \exists c > 0, n_0 \geq 1$ such that $cf(n) \leq T(n) \quad \forall n \geq n_0$

What Ω is actually for? O bounds a particular algorithm.
 Ω is how we bound a problem: "Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons"

21

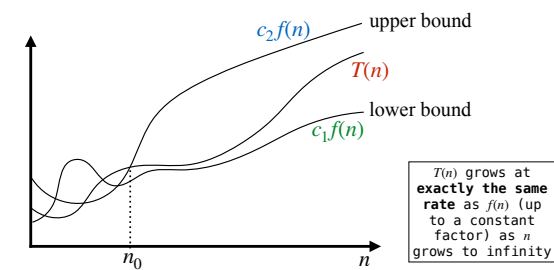
Q4-5

Which of the following is false?

- A. $4n + \log n$ is $\Omega(n)$
- B. $4n + \log n$ is $\Omega(\log n)$
- C. $4n + \log n$ is $\Omega(1)$
- D. $4n + \log n$ is $\Omega(n^2)$
- E. All of the above are true

22

Big Theta



$T(n)$ is $\Theta(f(n)) \iff T(n)$ is $O(f(n))$ and $T(n)$ is $\Omega(f(n))$

23

Q4-6

$T(n) = 3000n^3 + n^{3.5}$, which is the tightest correct statement?

- A. $T(n) = \Theta(n^3)$
- B. $T(n) = \Theta(n^{3.5})$
- C. $T(n) = \Theta(n^4)$
- D. $T(n) = O(n^3)$
- E. None of the above

24

Some useful rules

- Constants vanish
 - ✓ $cf(n) = \Theta(f(n))$, $c > 0$ so $\Theta(500n) = \Theta(n)$
- Sum rule
 - ✓ $O(f) + O(g) = O(\max(f, g))$ e.g. $\Theta(n^2) + \Theta(n) + \Theta(\log n) = \Theta(n^2)$
- Product rule
 - ✓ $O(f) \cdot O(g) = O(f \cdot g)$
- Transitivity
 - ✓ $f = O(g)$ and $g = O(h) \rightarrow f = O(h)$

25

Q4-7

- Simplify $T(n) = n(\log n + 40) + 5n$ to the tightest Θ

- A. $\Theta(n)$
- B. $\Theta(n \log n)$
- C. $\Theta(n^2)$
- D. $\Theta(n \log n + n)$
- E. None of the above

26

Tight vs loose bounds

- Every one of these is a true statement about linear search
 - ✓ $O(n)$ $O(n \log n)$ $O(n^2)$ $O(2^n)$ $O(n!)$
 - ✓ O is **only** an upper bound
- “what's the complexity?” always means the **tightest** bound you can justify
 - ✓ use Θ and the ambiguity disappears
- When people say “algorithm X is $O(n \log n)$ ” they almost always mean $\Theta(n \log n)$ for the worst case. Sloppy but universal (including in interviews and most textbooks)

27

Q4-8

- An algorithm's running time is exactly $T(n) = 4n + 7$. Which statement is false?

- A. $T(n) = O(n^2)$
- B. $T(n) = \Theta(n^2)$
- C. $T(n) = \Omega(1)$
- D. $T(n) = \Theta(n)$
- E. All of the above are true

28

Growth rates in practice

- **Asymptotic analysis** describes behavior for large n , reality also has small n
 - ✓ suppose algorithm A runs in $10^6 n$ operations and algorithm B in n^2 operations
 - asymptotically A wins, but **A is only faster once $n > 10^6$**
- Asymptotically slower algorithms may still be preferable, when they:
 - ✓ have much smaller constant factors
 - ✓ always operate on small inputs
 - ✓ are simpler to implement and get right
 - ✓ use less memory
 - ✓ offer better worst-case guarantees despite a worse average-case
- Takeaway
 - ✓ while asymptotic complexity matters for scalability, real-world performance depends on multiple factors!

29

Q4-9

- A student asks an AI assistant to solve 2-sum efficiently. It returns code for `two_sum_tp` (sort + two pointers) with this explanation:
 - ✓ “the optimal approach is to sort the array and use two pointers. This is $O(n \log n)$, which is the best possible for this problem since we must sort the input. The brute-force $O(n^2)$ approach should be avoided”
- The code is correct, and the complexity claim about that code is correct. What is wrong with the explanation?
 - A. Nothing, this is the optimal solution
 - B. The sort is actually $O(n^2)$, so the stated complexity is wrong
 - C. “Best possible” is false, 2-sum is solvable in expected $\Theta(n)$ with a hash table, sorting is not required
 - D. The two-pointer scan is incorrect on arrays containing negative numbers
 - E. None of the above

30

Q4-10

- Mark true if $T(n) = O(f(n))$

		$f(n)$			
		n^2	n^4	2^n	$\log n$
$T(n)$	$10^2 + 3000n + 10$				
	$21 \log n$				
	$500 \log n + n^4$				
	$\sqrt{n} + \log n^{50}$				
	$4^n + n^{5000}$				
	$3000n^3 + n^{3.5}$				
	$2^5 + n!$				

31

Q4-11

- Mark true if $T(n) = \Omega(f(n))$

		$f(n)$			
		n^2	n^4	2^n	$\log n$
$T(n)$	$10^2 + 3000n + 10$				
	$21 \log n$				
	$500 \log n + n^4$				
	$\sqrt{n} + \log n^{50}$				
	$4^n + n^{5000}$				
	$3000n^3 + n^{3.5}$				
	$2^5 + n!$				

32

Q4-12

• Mark true if $T(n) = \Theta(f(n))$

		$f(n)$			
		n^2	n^4	2^n	$\log n$
$T(n)$	$10^2 + 3000n + 10$				
	$21 \log n$				
	$500 \log n + n^4$				
	$\sqrt{n} + \log n^{50}$				
	$4^n + n^{5000}$				
	$3000n^3 + n^{3.5}$				
	$2^5 + n!$				
